

SECTION 5 COPROCESSOR PROGRAMMING

This section describes the guidelines for programming Freescale's floating-point coprocessors. The first portion of the section presents the guidelines for applications programming. It describes the concurrency with main processor instruction execution applicable to both coprocessors, and the coprocessor instruction concurrency provided by the MC68882 coprocessor. It also discusses the optimization of code for the MC68882 coprocessor.

The second portion of the section discusses systems programming considerations. It describes the state frame sizes, and lists the instructions required in the exception handlers for MC68881/MC68882 (FPCP) exceptions. The systems programming portion also describes the handling of exceptions by the MC68020/MC68030 (MPU) and FPCP combination, and code that detects and identifies the coprocessor.

This section primarily describes programming of the MC68882 coprocessor, since programs that run successfully in the MC68882 also run successfully in the MC68881. It is advisable to program for the MC68882 even if the MC68881 is currently being used, so that no program changes are required for upgrading to the MC68882.

5.1 APPLICATIONS PROGRAMMING

All applications programs that run successfully on the MC68881 can be used on the MC68882 without alteration, but optimization of code for the MC68882 provides significant reduction in execution time. The following paragraphs describe the concurrency available with the MC68881, the greater concurrency provided by the MC68882, and optimization techniques for MC68882 programs.

5.1.1 Concurrency

The M68000 coprocessor interface, the MC68020 and MC68030 microprocessors, and the MC68881 and MC68882 coprocessors are designed to provide the conventional sequential instruction execution models while instructions may actually be executed concurrently. Applications programs can be written with no provisions for concurrency; the system apparently executes the instructions in sequence. This apparent sequential execution is automatic, and the programmer need not be concerned about it.

5.1.1.1 CONCURRENT INTEGER AND FLOATING-POINT COMPUTATIONS. The M68000 coprocessor interface is designed to provide full support for the sequential instruction execution model. Although the M68000 coprocessor interface allows concurrency between coprocessor and main processor operations, the coprocessor must implement this concurrency while maintaining a programming model based on sequential instruction execution.

After the main processor initiates a floating-point instruction (by writing to the command CIR), it reads the response CIR. When the CA bit (bit [15] of the response CIR) is set, it

states that the main processor should perform the specified service and then read the response CIR again. The FPCP sets the CA bit to define portions of the floating-point instruction that cannot operate concurrently with main processor instruction execution. When the coprocessor can operate concurrently with main processor instructions, it clears the CA bit in the response primitive. Clearing the CA allows the main processor to proceed to the next instruction after it has read the response CIR and performed the specified service. This releases the main processor for concurrent operation. In the arithmetic floating-point instructions, the FPCP releases the main processor after the transfer phase is completed. Refer to **8.2 CONCURRENT INSTRUCTION EXECUTION** for further details of main processor/coprocessor concurrent instruction execution.

5

Within the boundaries of a floating-point instruction that does not allow concurrency with main processor instructions (response primitives return $CA=1$), the FPCP can allow the main processor to service pending interrupts. Bit [8] of the null primitive is the interrupts allowed (IA) bit, used by the FPCP to allow the main processor to check for pending interrupts and to service them before reading the response CIR again. This minimizes the worst case interrupt latency. Refer to **8.3 INTERRUPT LATENCY TIMES** for details.

As shown in Figure 5-1, once the MPU has initiated a floating-point instruction in the MC68881 and transferred the required operands to the coprocessor, the main processor is free to perform other instructions. Meanwhile, the coprocessor converts the operand to internal format, calculates the result, and rounds the result as required. The concurrency shown in Figure 5-1 for an MC68881 applies also to an MC68882.

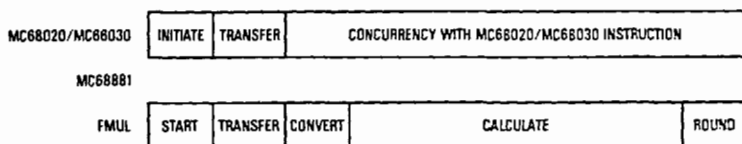


Figure 5-1. MC68881 Concurrency — FMUL Instruction

5.1.1.2 CONCURRENT FLOATING-POINT COMPUTATIONS. An FPCP arithmetic instruction with a floating-point data register destination releases the main processor when the coprocessor completes its transfer phase. If the next instruction is another floating-point instruction and if the main processor writes to the command CIR to initiate the next instruction while the APU is still busy with the previous instruction, the MC68881 returns a null ($CA=1$, $IA=1$) primitive in its response CIR. The MC68881 issues the null primitive because the APU can execute only one instruction at a time. Since the bus interface unit (BIU) cannot operate on any other instruction without the APU, no concurrency is possible. The MC68881 cannot begin to execute the second instruction, including the prefetch of necessary operands. The encoding of the response CIR remains unaltered until the instruction in the APU is completed. If the instruction terminates with an exception, a pre-instruction exception is taken.

With the MC68882, if the main processor initiates a second arithmetic instruction while a preceding instruction is executing in the APU as described in the preceding paragraph, the BIU transfers the instruction to the conversion unit (CU). Depending on the instruction, the

- The instructions that have operand data formats B, W, L, and P are listed in Table 5-1. When the CU receives an instruction with the operand data format of B, W, or L, it requests the BIU to transfer the necessary operand. Then, the CU waits for the APU to become idle so that it can hand off the instruction to the APU. If the instruction has an operand with data format P, the CU does not request the prefetch of the operand. In this case, it waits until the APU is idle to hand off the instruction to the APU.
- If the instruction is an FMOVE.X F_{Pm},F_{Pn} instruction, the CU does the following:
 - a. Releases the main processor.
 - b. Prefetches the source operand from F_{Pm}, unless the instruction currently operating in the APU uses F_{Pm} as a destination. In that case, the CU waits until the APU is idle before prefetching.
 - c. If the selected rounding precision is single or double, waits until the APU is idle and hands off the instruction to the APU.
 - d. If the operand data type is NAN, denormalized, or unnormalized, waits until the APU is idle and hands off the instruction to the APU.
 - e. If the instruction currently in the APU uses F_{Pn}, waits until the APU is idle before proceeding.
 - f. Writes the source operand into the destination floating-point data register without involving the APU.
- If the instruction is an FMOVE <ea>, F_{Pn} with an operand format of S, D, or X, the CU does the following:
 - a. Prefetches the source operand from memory by using the evaluate <ea> and transfer data primitive with CA = 0. This releases the main processor immediately after the source operand is written to the operand CIR.
 - b. Converts the memory source operand to internal extended format. If the selected rounding precision is single or double, waits until the APU is idle and hands off the instruction to the APU.
 - c. Creates a tag that represents the data type of the converted source operand (normalized, denormalized, zero, infinity, or NAN).
 - d. If the operand data type is NAN, unnormalized, or denormalized, waits until the APU is idle and hands off the instruction to the APU.
 - e. If the instruction currently in the APU uses F_{Pn}, waits until the APU is idle before proceeding.
 - f. Writes the converted source operand into F_{Pn} without involving the APU.
- If the instruction is an FMOVE F_{Pm},<ea> with a data format of S or D, the CU does the following:
 - a. Prefetches the source operand from F_{Pm}, unless the instruction currently operating in the APU uses F_{Pm} as a destination. In that case, the CU waits until the APU is idle before prefetching.
 - b. If the data type of the source operand is unnormalized, denormalized, or NAN, waits until the APU is idle and hands off the instruction to the APU.
 - c. Converts the source operand to the destination data format.
 - d. If the conversion results in an overflow or underflow, or if the INEX2 trap is enabled, waits until the APU is idle and hands off the instruction to the APU.
 - e. Creates a tag that represents the data type of the converted source operand (normalized, denormalized, zero, infinity, or NAN).

Transfers the converted operand to the memory destination (without involving the APU) by using the evaluate <ea> and transfer data primitive with CA=0. This releases the main processor immediately after the operand is read from the operand CIR.

- If the instruction is an FMOVE.X FPM,<ea>, the CU does the following:
 - a. Prefetches the source operand from FPM, unless the instruction currently operating in the APU uses FPM as a destination. In that case, the CU waits until the APU is idle before prefetching.
 - b. If the data type of the source operand is unnormalized, denormalized, or NAN, waits until the APU is idle and hands off the instruction to the APU.
 - c. Transfers the converted operand to the memory destination (without involving the APU) by using the evaluate <ea> and transfer data primitive with CA=0. This releases the main processor immediately after the operand is read from the operand CIR.
- If the instruction is listed in Table 5-4 and if the source (FPM) and destination (FPn or FPC:FPs) are all floating-point data registers, the CU does the following:
 - a. Releases the main processor.
 - b. Prefetches the source operand from FPM if possible.
 - c. Waits until the APU is idle and hands off the instruction to the APU.
- If the instruction is listed in Table 5-4 and if the source is external to the MC68882, the CU does the following:
 - a. Prefetches the source operand from memory by using the evaluate <ea> and transfer data primitive with CA=0. This releases the main processor immediately after the source operand is written to the operand CIR.
 - b. If the source operand format is single or double precision, converts the source operand to the extended precision internal format.
 - c. Creates a tag that represents the data type of the converted source operand (normalized, unnormalized, denormalized, zero, infinity, or NAN).
 - d. Waits until the APU is idle and hands off the instruction to the APU.

Table 5-1 lists the minimum-concurrency instructions. The monadic operations, designated <mop> in Tables 5-1 and 5-4, are listed in Table 5-2. The dyadic operations, designated <dop> in Tables 5-1 and 5-4, are listed in Table 5-3. Table 5-4 lists the partially-concurrent instructions, and Table 5-5 lists the fully-concurrent instructions.

The instructions that have external operands and are listed in Tables 5-4 and 5-5 prefetch the source operand (if no register conflict exists) and release the main processor after the operand is transferred. When a third instruction is received in the command CIR while the APU is busy and the CU is either busy or waiting to hand off its instruction to the APU, the third instruction must wait. The BIU encodes a null (CA=1, IA=1) primitive in the response CIR until the CU becomes idle or hands off its instruction to the APU. Note that this situation is analogous to the situation in the MC68881 when a second instruction is received in the command CIR while the APU is still busy with a previous instruction. The difference is that the MC68881 waits until the APU is available, while the MC68882 waits until the CU is available.

The conditional instructions listed in Table 5-6 do not allow concurrency. These instructions are not executed unless both the CU and the APU are idle and all exception flags are cleared. An extreme case occurs if the MPU writes to the condition CIR of the MC68882

Table 5-1. Minimum-Concurrency Instructions

Instruction	Operand Syntax	Operand Format
FMOVE	<ea>,FPn FPm,<ea> FPm,<ea>{#k} FPm,<ea>{Dn} <ea>,FPcr FPcr,<ea>	B,W,L,P B,W,L P P L L
FMOVECR	#ccc,FPn	X
FMOVEM	<ea>,<list> <ea>,Dn <list>,<ea> Dn,<ea>	L,X X L,X X
FTST	FPm	B,W,L,P
F<mop>	<ea>,FPn	B,W,L,P
F<dop>	<ea>,FPn	B,W,L,P
FSINCOS	<ea>,FPc:FPs	B,W,L,P

Table 5-2. Monadic Instructions

Instruction	Function
FABS	Absolute Value
FACOS	Arc Cosine
FASIN	Arc Sine
FATAN	Arc Tangent
FATANH	Hyperbolic Arc Tangent
FCOS	Cosine
FCOSH	Hyperbolic Cosine
FETOX	e^x
FETOXM1	$e^x - 1$
FGETEXP	Extract Exponent
FGETMAN	Extract Mantissa
FINT	Extract Integer Part
FINTRZ	Extract Integer Part, Rounded-to-Zero
FLOGN	$\ln(x)$
FLOGNP1	$\ln(x + 1)$
FLOG10	$\log_{10}(x)$
FLOG2	$\log_2(x)$
FNEG	Negate
FSIN	Sine
FSINH	Hyperbolic Sine
FSQRT	Square Root
FTAN	Tangent
FTANH	Hyperbolic Tangent
FTENTOX	10^x
FTWOTOX	2^x

Table 5-3. Dyadic Operations

Instruction	Function
FADD	Add
FCMP	Compare
FDIV	Divide
FMOD	Modulo Remainder
FMUL	Multiply
FREM	IEEE Remainder
FSCALE	Scale Exponent
FSGLDIV	Single Precision Divide
FSGLMUL	Single Precision Multiply
FSUB	Subtract

5

Table 5-4. Partial-Concurrency Instructions

Instruction	Operand Syntax	Operand Format
FTST	<ea> FPm	S,D,X X
F<mop>	<ea>,FPn FPm,FPn	S,D,X
F<dop>	<ea>,FPn FPm,FPn	S,D,X
FSINCOS	<ea>,FPc:FPs FPm,FPc:FPs	S,D,X X

Table 5-5. Fully-Concurrent Instructions

Instruction	Operand Syntax	Operand Format	Degraded to No Concurrency	Degraded to Partial Concurrency
FMOVE	FPm,FPn	X	a	b,c,f
FMOVE	<ea>,FPn	S,D		b,c,f
FMOVE	<ea>,FPn	X		b,c,f
FMOVE	FPm,<ea>	S,D	a	b,d,e
FMOVE	FPm,<ea>	X	a	b

- a. Register Conflict of FPm with preceding instruction's destination floating-point data register
- b. NAN, Unnormalized or Denormalized Data Types
- c. Rounding Precision in FPCR set to Single or Double
- d. INEX2 bit in FPCR EXC byte is enabled
- e. An Overflow or Underflow occurs
- f. Register conflict of FPn with preceding instruction's destination floating-point data register

both the CU and APU are busy, the appropriate exceptions are enabled, and the exceptions in the CU and APU each report an exception. The MC68882 reports these exceptions, one at a time, until both exception handlers have been executed. As a consequence, the conditional instruction is re-started twice to ensure that the reported condition codes contain information reflecting the result of all previous instructions and related exception handlers. Therefore, a sequential execution model can be guaranteed. It is possible that a BSUN exception is reported by the conditional instruction, but the BSUN exception would be reported long after the instructions and related exceptions have been executed and completed.

Table 5-6. Conditional Instructions

Instruction	Operand Syntax	Operand Size or Format
FBcc	<label>	W,L
FDBcc	Dn,<label>	W
FNOP	None	None
FSc	<ea>	B
FTRAPcc	None #xxx	None W,L

Consider the case of two consecutive FMUL instructions followed by an FMOVE instruction, as shown in Figure 5-2. The following assumptions apply:

1. Both FMUL instructions have external operands (opclass 010),
2. The FMOVE instruction has a memory destination (opclass 011), and
3. No exceptions are enabled.

The main processor initiates the second FMUL instruction, and the MC68881 returns the null (CA=1) primitive as long as the APU is involved in the calculate phase of the first FMUL instruction. When the APU becomes available, the BIU returns the evaluate effective address and transfer operands (CA=0) primitive and begins the transfer phase of the second FMUL instruction. At this point, since the CA bit is clear, the main processor begins the execution of another instruction while the MC68881 converts the operand to internal format and begins the calculate phase of the second FMUL instruction. Since the next MPU instruction is an FMOVE instruction, the MPU initiates the FMOVE instruction, but the MC68881 returns the null (CA=1) primitive until the second FMUL instruction completes.

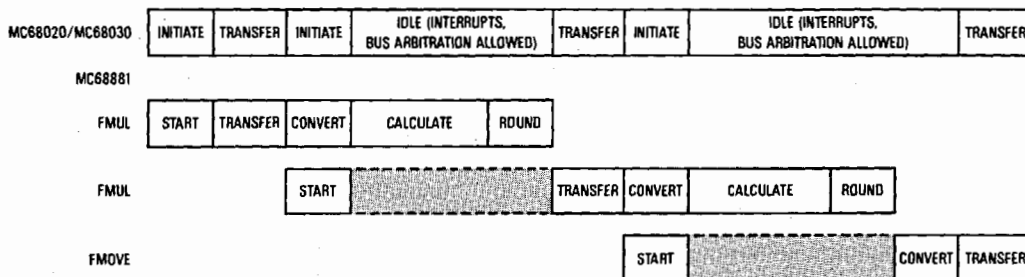


Figure 5-2. MC68881 Concurrency — FMUL Followed by FMUL and FMOVE

In this example, the MPU continues to examine the response CIR of the MC68881 while it completes each of the two FMUL instructions. Should an interrupt occur during these times, the main processor services the interrupt but does not initiate any other instruction.

The MC68882 can execute floating-point instructions concurrently by performing conversions between external binary real data formats (S, D, and X) and the internal extended format in the conversion unit (CU) while the arithmetic processing unit (APU) is calculating the result of a preceding instruction. Additional concurrency is provided by making the floating-point data registers accessible to both the CU and APU simultaneously.

5

Figure 5-3 shows the same three floating-point instructions executing in an MC68882. As soon as the operand of the first FMUL instruction has been transferred to the coprocessor, the main processor begins executing the next instruction, another FMUL instruction with an external source operand. Provided the operand is a binary real operand and no register conflict occurs, the coprocessor can transfer and convert the operand while it continues to calculate the product of the first FMUL instruction. As soon as the operand transfer completes, the main processor begins executing the FMOVE instruction. Since the CU contains the converted operand for the second FMUL instruction at this time, it is not available to convert the source operand of the FMOVE instruction. However, when the APU completes the rounding phase for the first FMUL instruction, it accepts the operand from the CU and begins calculations for the second FMUL instruction. The CU now converts the source operand of the FMOVE instruction to the destination format. The bus interface unit (BIU) transfers the converted operand to the external destination, completing the second FMUL instruction.

The effect of the concurrency provided by the MC68882 is to execute three instructions during a time period equal to the execution time of the first instruction plus the computation time of the second instruction. Execution of the third instruction is completely overlapped by the second instruction.

In this example, execution of the second FMUL instruction is partially concurrent with execution of the first FMUL instruction, and execution of the FMOVE instruction is fully concurrent with execution of the second FMUL instruction. Some MC68882 instructions do not execute concurrently. Others execute partially concurrently, and some execute with full concurrency. However, little concurrency is possible when the operand is in integer or packed decimal format.

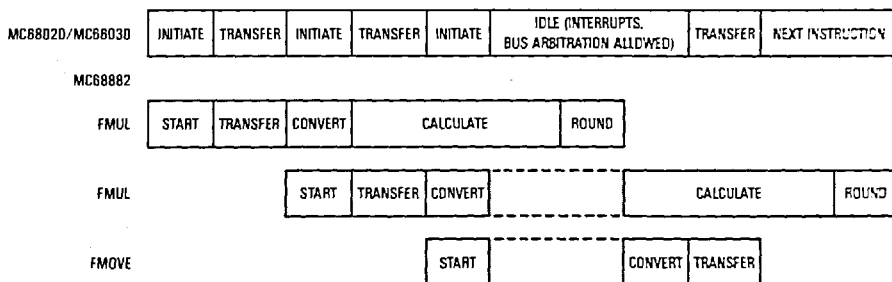


Figure 5-3. MC68882 Concurrency — FMUL Followed by FMUL and FMOVE

A program that runs successfully on the MC68881 runs on the MC68882 with improved performance. However, the code can be optimized to exploit the features of the MC68882 for the maximum performance improvement. Optimization requires the following steps:

1. Unroll any rolled loops to obtain at least a 2x unrolled version,
2. Eliminate register conflicts by rearranging FMOVE instructions, and
3. Rearrange FMOVE instructions so that the fastest FMOVE instructions follow the fastest arithmetic instructions, and the slowest FMOVE instructions follow the slowest arithmetic instructions.

5.1.2.1 UNROLLING LOOPS. A rolled loop consists of the instructions to perform the operations of the loop once using a single index value during each iteration. The performance of the MC68882 is improved by unrolling the loop, so that an iteration performs those operations more than once, using two or more index values. The recommended 2x unrolled version performs the operations twice.

The rolled version of a loop allows little optimization; a register conflict is inevitable. The 2x unrolled version can use different floating-point data registers for each repetition of the instructions. The FMOVE instructions can be placed in the optimum locations.

5.1.2.2 AVOIDING REGISTER CONFLICTS. The following rules define conflicts between floating-point data registers.

- A register conflict occurs when the destination register of an instruction is the source register of the following instruction, and that instruction is a fully-concurrent instruction listed in Table 5-5. For example:
 FADD.D (ea),FP0
 FMOVE.D FP0,(ea) FP0 conflicts
- A register conflict occurs when the destination register of an instruction is the destination register of the following instruction, and that instruction is a fully-concurrent instruction listed in Table 5-5. For example:
 FADD.D (ea),FP0
 FMOVE.D (ea),FP0 FP0 conflicts
- No other combination of source and destination registers of two consecutive instructions cause a register conflict.

The second case (where an FMOVE instruction uses the same destination register as the preceding instruction) is an unlikely case, since the result of the first instruction is lost. However, the MC68882 provides the same result as the MC68881 even for this case.

5.1.2.3 ARRANGING FMOVE INSTRUCTIONS. The FMOVE instruction is fully concurrent when the operands are in binary real data format, no register conflicts exist, and the notes of Table 5-5 do not apply. However, the execution time of the FMOVE instruction is hidden completely only when the overlap time of the preceding instruction exceeds the execution time of the FMOVE instruction. Thus, the fastest FMOVE instructions should follow the fastest arithmetic instructions, FADD, for example. Also, the slowest FMOVE instructions should follow the slowest arithmetic instructions, such as FMUL. Refer to the tables of

Table 5-7. FMOVE Instruction Execution Times

Operand	Type	Execution Time (Clocks)
FMOVE.X	FPx,FPy	21
FMOVE.D	<ea>,FPy	39
FMOVE.D	FPy,<ea>	55

5.1.2.4 PERFORMANCE IMPROVEMENT EXAMPLE. The DAXPY subroutine inner loop of the Linpack benchmark (Linpack loop) is an appropriate example for illustrating optimization for the MC68882. Figure 5-4 shows the source code for the rolled version of the Linpack loop.

```

*          VECTOR TIMES A CONSTANT PLUS A VECTOR
*          X(i) = Y(i)*C + X(i)

          MOVE.L    #count,D0
          FMOVE.D   <ea,C>,FP0

LOOPTOP   FMOVE.X   FP0,FP1
          FMUL.D   <ea,Y(i)>,FP1
          FADD.D   <ea,X(i),FP1
          FMOVE.D   FP1,<ea,X(i)
          DBRA     D0,LOOPTOP

```

Figure 5-4. Rolled Version of Linpack Loop

Optimization of this code for the MC68882 consists of unrolling the loop, and rearranging the FMOVE instructions. Notice that FP1 contains the result of the computations using index i, and that FP2 contains the result of the computations using index i + 1. Also notice that the two FMOVE instructions that move registers to registers are executed following FADD instructions, and that the FMOVE instructions that move registers to effective addresses are executed following FMUL instructions. Figure 5-5 shows the source code for the optimized Linpack loop.

5.2 SYSTEMS PROGRAMMING

The guidelines for systems programming relate to exception processing. The sizes of the state frames stored by exception handlers are discussed first. Next, the section discusses the FSAVE, BSET, and FRESTORE instructions required in exception handlers. Then, the handling of specific exceptions is discussed. Code that detects the presence of a floating-point coprocessor and identifies the coprocessor is also discussed.

5.2.1 State Frame Sizes

The sizes of the state frames stored by the FSAVE instruction differ for the MC68882 and MC68881, as shown in Table 5-8.

```

*      VECTOR TIMES A CONSTANT PLUS A VECTOR
*      X(i) = Y(i)*C + X(i)

      MOVE.L      #count,D0
      FMOVE.D     <ea_C>,FP0
      FMOVE.X     FP0,FP1
      FMUL.D      <ea_Y(i)>,FP1
      BRA         LOOPSTRT

LOOPTOP  FMOVE.X     FP0,FP1
         FMUL.D      <ea_Y(i)>,FP1
         FMOVE.D     FP2,<ea_X(i+1)>
LOOPSTRT FADD.D      <ea_X(i),FP1

         FMOVE.X     FP0,FP2
         FMUL.D      <ea_Y(i+1)>,FP2
         FMOVE.D     FP1,<ea_X(i)>

         FADD.D      <ea_X(i+1)>,FP2

         DBRA        D0,LOOPTOP

         FMOVE.D     FP2,<ea_X(i+1)>

```

Figure 5-5. Optimized Linpack Loop

Table 5-8. State Frame Sizes

Device	Null Frame	Idle Frame	Busy Frame
MC68881	4 Bytes	28 Bytes	184 Bytes
MC68882	4 Bytes	60 Bytes	216 Bytes

The size of the null state frame is four bytes for both coprocessors. The size of the other state frames is 32 bytes larger for the MC68882 than for the MC68881. The MC68882 uses the additional bytes to store the state of the conversion unit.

5.2.2 Exception Handler Code

The code for floating-point exception handlers for the MC68882 must include the following instructions:

1. An FSAVE instruction at the beginning of the handler (ahead of the first coprocessor instruction)
2. A BSET or similar instruction following the FSAVE instruction to set the EXC_PEND flag (bit 27) of the BIU flag in the idle state frame
3. An FRESTORE instruction immediately preceding the RTE instruction

Handlers for the following exceptions require these instructions even if the handlers contain no floating-point instructions:

Branch or Set on Unordered Condition	Operand Error
Inexact Result	Overflow
Floating-Point Divide by Zero	Signaling NAN
Underflow	

Handlers for interrupts, F-line emulation, FTRAPcc instructions, and other exceptions must set the EXC_PEND bit in the BIU flag long word, but any exception handler that contains one or more floating-point instructions must begin with an FSAVE instruction and have an FRESTORE instruction preceding the RTE instruction. No requirements are imposed on the floating-point protocol violation exception because it is considered to be a catastrophic exception from which no recovery is possible.

When a floating-point exception handler that does not begin with an FSAVE instruction executes in a system that uses an MC68882 coprocessor, one of two things happens. Either the next MC68882 instruction takes the same exception, producing an infinite loop, or it takes a protocol violation exception.

When a floating-point exception handler that begins with an FSAVE instruction but does not set the EXC_PEND bit executes in a system that uses an MC68882 coprocessor, the next MC68882 instruction takes the same exception, also producing an infinite loop.

5

When a floating-point exception handler that begins with an FSAVE instruction but does not end with an FRESTORE instruction is executed in a system that uses an MC68882 coprocessor, a partially-executed instruction following the exceptional instruction may never be completed. Figure 5-6 shows the required instructions in a minimum exception handler for an MC68882.

HANDLER	FSAVE	-(SP)	SAVE INTERNAL STATE
	MOVE.B	(SP),D0	FIRST BYTE OF STATE FRAME
	BEQ	NULL	BRANCH IF NULL FRAME
	CLR.L	D0	CLEAR DATA REGISTER
	MOVE.B	1(SP),D0	LOAD STATE FRAME SIZE
	BSET	#3,(SP,D0)	SET BIT 27 OF BIU
	.		
	.		
	.		
NULL	FRESTORE	(SP)+	RESTORE STATE
	RTE		RETURN

Figure 5-6. Minimum Exception Handler

An exception handler can access the idle state frame to obtain information about the exception. The offsets of the exceptional operand, the operand register, and the BIU flags are different in the MC68881 and the MC68882. In the MC68882, these offsets are greater than those in the MC68881 by \$20. For example, the offset for the exceptional operand is \$08 in the MC68881 and \$28 in the MC68882. However, the negative offsets (from the bottom of the state frame) are the same for both coprocessors. Figure 5-7 shows a code fragment that can be used to access the exceptional operand and the operand register image in an exception handler for either coprocessor.

5.2.3 Processing of Special Conditions

The designs of the MPU, the M68000 coprocessor interface, and the FPCP provide the performance benefits of concurrent operation while maintaining a conventional sequential instruction execution model. Processing of special conditions is also performed as if instructions were executed sequentially. Refer to the coprocessor interface section of the appropriate microprocessor user's manual for additional information.

MOVE.B

1(SP),D0

LOAD FRAME LENGTH INTO D0

MOVE.L
MOVE.LXOPER(SP,D0),(ea)
OPREG(SP,D0),(ea)ACCESSES THE FIRST LONGWORD OF THE EXCEPTIONAL OPERAND
ACCESSES THE OPERAND REGISTER IMAGE

Figure 5-7. Idle State Frame Access Example

5.2.3.1 INTERRUPTS. The main processor can process interrupts at any instruction boundary and during the execution of a general or conditional category coprocessor instruction under either of two conditions. When the main processor receives a null (CA = 1, IA = 1) primitive, the MPU services any pending interrupts prior to reading the response CIR. The MPU also services pending interrupts when the trace exception is enabled and the MPU receives a null (CA = 0, IA = 1, PF = 0) primitive.

The MPU uses the ten-word mid-instruction stack frame shown in Figure 7-16 when it services interrupts during the execution of a general or conditional category coprocessor instruction. Using this stack frame allows the MPU to perform all necessary processing and return to read the response CIR. Thus, the MPU services the interrupt while the FPCP continues to execute the coprocessor instruction.

During execution of an FSAVE instruction, when the MPU reads the not ready format word, it also services interrupts. After servicing any pending interrupts, the MPU returns and reinitiates the FSAVE instruction.

5.2.3.2 BUS ARBITRATION. During execution of a floating-point instruction, the MPU can relinquish control of the bus through bus arbitration. If the FPCP has released the MPU and is completing execution of the instruction, relinquishing the bus has no effect on the coprocessor. If the MPU is involved in a dialog with the coprocessor, relinquishing the bus delays the execution of the instruction in the FPCP. However, since the coprocessor communicates with the MPU by placing a response primitive in the response CIR for the MPU to read, no adverse effect occurs. The only effect of the bus arbitration is a longer delay while the coprocessor awaits the services of the MPU.

5.2.3.3 CONTEXT SWITCHING. In a multi-tasking environment, the context of the FPCP may be changed asynchronously with respect to coprocessor operations. The coprocessor may be interrupted at any point during the execution of an instruction. The FSAVE and FRESTORE instructions are used to save and restore the context of the coprocessor during context switches.

An FSAVE instruction stops execution of the instruction in the coprocessor at the earliest interruptable point, and stores the state of the coprocessor. The coprocessor is now available to the program executing in the new context. When the interrupted program resumes, an FRESTORE instruction loads the saved state of the coprocessor, restoring the coprocessor to its previous state. The coprocessor continues from the point at which it was interrupted.

tate frames defined for the null, idle, and busy states of the coprocessor contain all information the coprocessor requires to resume operation. Inclusion of the coprocessor version number in the format word and the checking of that version number during execution of the FRESTORE instruction prevent restoration of an incompatible context (e.g., an MC68881 context in an MC68882).

5.2.3.4 BUS ERRORS. A bus error can occur during initiation of a coprocessor instruction or while the MPU is accessing memory or CPU address space during execution of a coprocessor instruction. A bus error during initiation of an instruction is used as an indication that the coprocessor is not present, and the MPU takes an F-line emulator exception. A bus error during a memory access indicates that some fault (e.g., parity error or page fault) prevents the memory system from providing the requested operand. The coprocessor interface, being asynchronous, does not require the MPU to service the bus error exception at once. No time restrictions on the main processor's response to a bus error exception exist. After the exception handler has corrected the cause of the bus error, the MPU returns to the point in the coprocessor instruction dialog at which the fault occurred.

5.2.3.5 EXCEPTION PROCESSING. During the execution of a coprocessor instruction, the coprocessor releases the main processor after the main processor has completed all the services the coprocessor requires to execute the instruction. Any exception processing the main processor performs after being released and before initiation of another coprocessor instruction has no effect on the coprocessor.

Either the main processor or the coprocessor can detect an exception during execution of a floating-point instruction. The handlers for these exceptions are bracketed with FSAVE and FRESTORE instructions as previously described to ensure that coprocessor state information about concurrently executing instructions is properly restored after execution of the exception handler completes.

5.2.3.6 SIMULTANEOUS FLOATING-POINT EXCEPTION AND TASK SWITCH INTERRUPT.

Since an interrupt signal can occur at any time, a task switch interrupt can occur simultaneously with a floating-point exception detected by the coprocessor. The FPCP and the coprocessor interface with the MPU are designed to preserve the sequential instruction execution model in this case. Figure 5-8 shows an FMUL instruction executing in an MC68882, followed by an FADD instruction. A task switch interrupt occurs as the main processor responds to the exception. The sequence of events is as follows:

1. The MC68882 is executing the two instructions concurrently.
2. The FMUL instruction is executing in the APU, the CU has performed the conversion of the source operand, and the CU is waiting to hand off the FADD instruction when the APU becomes idle. The MC68882 is returning null (CA=0, IA=0) primitives to synchronize the main processor. The main processor is reading the primitives, responding to any pending interrupts.
3. The FMUL instruction detects an exception, which is reported by the FADD instruction with a take mid-instruction exception primitive.
4. The main processor recognizes a pending interrupt as it reads the take mid-instruction exception primitive. Because of the internal timing, however, the MPU processes the floating-point exception first.

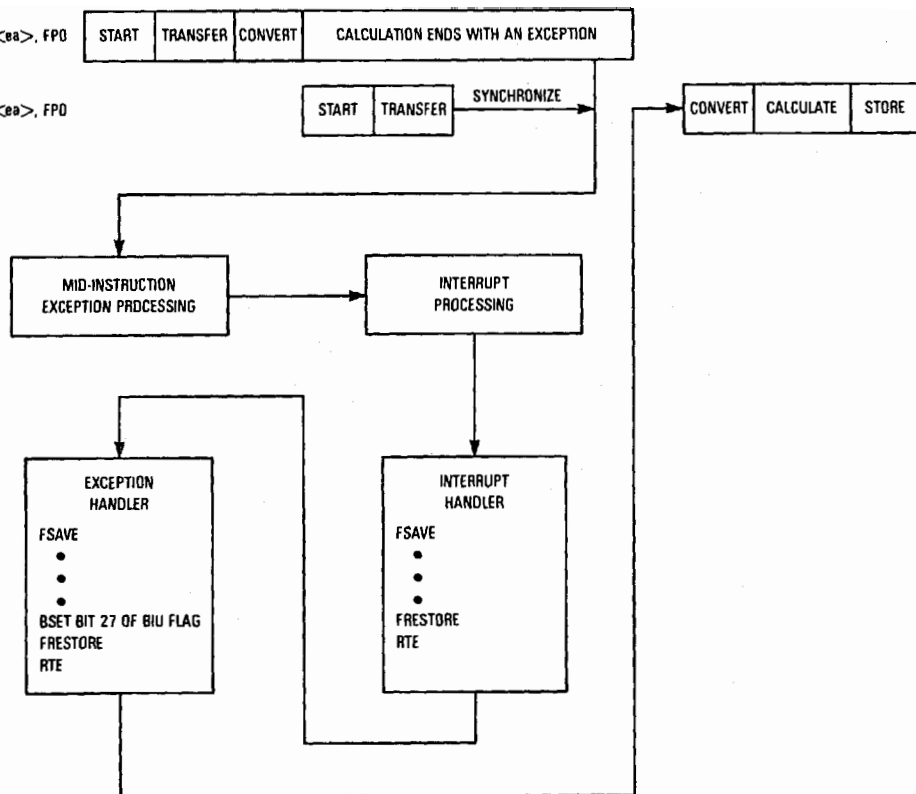


Figure 5-8. Simultaneous Task Switch Interrupt and Floating-Point Exception

5. The processing of the exception completes, and the main processor begins processing the interrupt before executing the first instruction in the floating-point exception handler.
6. The main processor executes the interrupt handler. Because the interrupt handler does not contain a BSET instruction that sets bit 27 of the BIU flag word, the state restored by the FRESTORE instruction in the handler indicates that the floating-point exception has not been serviced. The FADD instruction is not allowed to continue.
7. The floating-point exception handler is executed. This handler includes a BSET instruction that sets bit 27 of the BIU flags word. When the FRESTORE instruction restores the state frame, the FADD instruction continues.

In this example, if the interrupt handler allowed the FADD instruction to continue, the FADD instruction would have overwritten the contents of FP0, and the results would have been incorrect. The MC68882 exception model handles this worst-case situation correctly.

5.2.4 Detecting Coprocessor Presence

A program or an exception handler may need to know if a floating-point coprocessor is available, or which type coprocessor is present. The code fragment in Figure 5-9, which executes at the supervisor privilege level, detects and identifies the coprocessor.

	CLR.B	FLAG	CLEAR NO PROCESSOR FLAG
	FNOP		DETECT COPROCESSOR (SEE NOTE)
	MOVE.B	FLAG,D0	LOAD FLAG
	BNE	NOCOP	NO COPROCESSOR BRANCH
	FSAVE	—(SP)	SAVE INTERNAL STATE
	CLR.L	D0	ZERO INDEX
	MOVE.B	1(SP),D0	OBTAIN STATE FRAME SIZE
	CMPI	#\$18,D0	MC68881?
	BEQ	ONE	YES
	.		CODE FOR MC68882
	.		
	.		
	BRA	START	END OF MC68882 CODE
ONE	.		CODE FOR MC68881
	.		
	.		
START	...		START OF CODE COMMON TO BOTH COPROCESSORS

NOTE: When no coprocessor is present, an exception handler executes at this point. See text.

Figure 5-9. Coprocessor Identification Code

The FNOP instruction takes an F-line emulation exception when no floating-point coprocessor is available. The F-line emulation exception handler must set the no coprocessor flag and increment the stacked PC value by four. The BNE instruction branches around this code when no coprocessor is present. The instructions immediately following the BEQ instruction are executed for an MC68882 coprocessor; those at label ONE are executed for an MC68881.